SAMS Kernel

Music and sound: the player, the compiler, the song format

Version 1.25.0 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

1. The sound chip in brief

2. Playing a song

3. The compiler

4. The song format (SKM)

5. Try it

Decision: the default notation will be TXA's `.txm` (2 November 2026)

The kernel plays music through the TI's sound chip from songs held in SAMS pages. A song is a stream of chip commands, made on the PC by `tools/skmusic.py` from a **MIDI file** or a **text score**; the player steps it **once a frame** from the kernel's main loop, so an application starts a song with one call and then forgets it. This document covers the chip, the calls, the compiler, the text score language, the song format, and what I did and did not check.

What I checked, and what I could not. I cannot listen. The tests check what the player does to the chip: the bytes it writes and when (against what the compiler produced), the pitch of what a model of the sound chip makes of those writes (a Fourier transform of the synthesised sound), pause, resume, looping and stepping from `KIDLE`. The compiler's choices of notes and the model of the noise generator are approximations you should *hear* before you trust them: `skmusic.py --wav` writes a preview, and the real test is a real TI.

Routes. Two ways to work: [PC] / [TI] / [PC or TI]. [PC] is a PC with Python 3 and xdt99 (build with `skbuild.py`, run in the simulator or js99er); **[TI]** is the TI itself with the workbench cartridge (editor, assembler, launcher); **[PC or TI]** is the same either way. The full table is in PROGRAMMERS-GUIDE.md part 0. Here: **compiling** songs (`skmusic.py`, the `"from"` entry in `app.json`) and putting them into SAMS pages is **[PC]**; **playing** one (`MUPLAY`, `MUPAUSE`, `MUSTOP`) is **[PC or TI]** code but needs the song in pages, so it runs from a cartridge built on the **[PC]**; **tones** (`SNDTONE`, `SNDOFF`) are **[PC or TI]** and are the music route on the **[TI]**: `programs/beep.a99` on the workbench disk plays a note on each of keys 1-8 (assemble it with A, run it with R). The simulator checks and the WAV preview are **[PC]**.

1. The sound chip in brief

The TMS9919 (SN76489 family) has **three tone voices and one noise voice**. You write it a byte at a time at `SOUND` (`>8400`):

Byte	Meaning
`1 cc 0 dddd`	voice `cc` (0-2): the low 4 bits of its divider. A byte `0 dddddd` then follows with the high 6 bits
`1 cc 1 aaaa`	voice `cc` (0-3): attenuation, 0 loudest, 15 off, 2 dB a step
`1 11 0 0 f nn`	the noise voice: `f` = 1 white, 0 periodic; `nn` = 0, 1, 2 shift rate high, medium, low, 3 follows voice 2's tone

A tone's frequency is **111,861 / divider** Hz (3,579,545 / 32), with a 10-bit divider (1-1023): the lowest note is about 109 Hz (A2), and the high notes get coarse. Dividers for the notes of equal temperament:

oct	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
2	-	-	-	-	-	-	-	-	-	1017	960	906
3	855	807	762	719	679	641	605	571	539	508	480	453
4	428	404	381	360	339	320	302	285	269	254	240	226
5	214	202	190	180	170	160	151	143	135	127	120	113
6	107	101	95	90	85	80	76	71	67	64	60	57
7	53	50	48	45	42	40	38	36	34	32	30	28

(Octave 4 is middle C's octave; A4 is 254, 440.4 Hz. The chip has no envelope: a note's volume changes only when you write it.)

2. Playing a song

[PC] (the `app.json` entry and `skbuild.py`); the calls are **[PC or TI]** code.

Put the song in the build, then play it:

```
        "data": [
          {"file": "tune.skm", "from": "songs/tune.mml", "loop": true},
          {"file": "jingle.skm", "from": "songs/jingle.mid"}
        ]
```

The build compiles each `from` file with `skmusic.py` (a `.mml` score or a `.mid` MIDI file), puts the result in SAMS pages and defines an equate with the song's first page: `PD_TUNE`, `PD_JINGLE` (the file name in capitals). `"loop": true` repeats the song from its start; `"loop_at": 4.0` repeats from that moment (an introduction plays once); `"fps": 50` for a PAL console; `"volume"` and `"transpose"` adjust every note.

```
        LI   R0,PD_TUNE           the song's first SAMS page
        BL   @MUPLAY              R0 = 0 it began, 1 those pages hold no song
```

Routine	In	Does
`MUPLAY`	R0 = the song's first page	starts the song (replacing any playing); R0 = 0, or 1 if the pages do not hold a song
`MUSTOP`		stops the song and silences the chip
`MUPAUSE`	R0 = 1 pause, 0 resume	pause silences the chip; resume puts back each register's last value, so the song carries on in tune
`MUTICK`		one frame of the song: **the kernel calls it for you**; call it yourself only if you run your own frame loop
`SNDTONE`	R0 = voice 0-2, R1 = divider, R2 = attenuation	sets a tone voice directly: sound effects
`SNDOFF`		silences the four voices (song or no song)

Variables you may read: `MUSTAT` (non-zero while a song plays), `MUPAUS` (non-zero while paused), `MUPOS` (the offset of the next command in the song). Do not write them.

How it is stepped. `MUTICK` runs once a frame in the main loop, just after `VSYNC`, and also each time `KIDLE`, `KPFR` or `KWAIT` sees a frame go by. So the music goes on while your `TICK` does its work, and also **through long operations that poll the keyboard** (reading a file, a slow redraw) provided they call `KIDLE` between steps. Work that neither returns to the main loop nor calls `KIDLE` stalls the music; and a `TICK` that takes more than a frame slows it, because the song moves one step a main-loop pass, not by the clock. Music runs at the console's frame rate (60 Hz, or 50 on a PAL console: tell the compiler with `"fps": 50`).

Cost (measured in the simulator, tune playing; the paging routines it calls are included). A frame in which the song is only holding costs 8 instructions. A frame in which the chip is written costs about 120 (median; 90% under 150, the busiest 240), roughly 70 of them mapping the song's pages and putting the application's mapping back; against a frame's budget of a few thousand instructions that is a few percent. In that tune a third of the frames write the chip. The player and its tables are 192 bytes. It maps the song through slot B (`MAPE`, `MAPF`) and restores slot B each time, so it never disturbs the pages the application has mapped.

Sound effects. `SNDTONE` and raw writes to `SOUND` share the chip with the song: a song's next change to a voice overwrites an effect on it. Give effects a voice the song leaves alone (compile the song with fewer voices), or play effects while the song is paused. The console's own sound-list player (the interrupt routine plays lists from VRAM or GROM) still works as before, and shares the chip in the same way; the player and the lists have not been mixed in a test.

3. The compiler

[PC] only: `skmusic.py` is Python.

```
python3 tools/skmusic.py tune.mml -o tune.skm --loop --wav tune.wav --info
```

Option	Meaning
`--fps 60|50`	the console's frame rate
`--loop`, `--loop-at SECONDS`	play over and over from the start / from that moment (the loop point is written out in full, so the voices are right when it comes round)
`--volume N`	add N (plus or minus) to every note's volume, 0-15
`--transpose N`	move every note by N semitones
`--wav FILE`	a preview of the song made by a model of the chip (22,050 Hz mono)
`--info`	print the size, length and note count

From MIDI. Notes of every track are shared among the three tone voices: the highest and the lowest note sounding always get one, and an inner note takes the third when it is free (a note that cannot be placed is dropped, and the count is reported). A note's volume follows its velocity. Notes lower than 110 Hz are moved up by octaves (and very high ones down) so the chip can play them. The **drum channel (10)** goes to the noise voice: kicks, snares and hi-hats become short noise bursts that fade out. Tempo changes are followed. Only standard MIDI files with ticks-per-beat timing are read (not

SMPTE).

From a text score (MML). One track to a voice; `;` starts a comment:

```
; "Ode to Joy": examples/music/songs/tune.mml
V1: t132 o4 l4 e e f g g f e d c c d e e.  l8 d l2 d
V2: o3 l2 c g c g c g g g
V3: o2 l1 c c c g
N:  l8 k6 r k4 r k5 r k4 r
```

Command	Meaning
`V1:` `V2:` `V3:` `N:`	the track: a tone voice, or the noise voice. A track may continue on later lines
`tN`	tempo in quarter notes a minute (default 120)
`oN`, `<`, `>`	octave (o4 c is middle C); one octave down; one up
`lN`	the default length: 1 whole, 2 half, 4 quarter, 8 eighth, 16, 32
`vN`	volume 0-15 (default 12)
`qN`	gate 1-8: the note sounds N/8 of its length, then a gap (default 7)
`c d e f g a b`	a note; then `+` or `#` sharp, `-` flat, then a length (`c4`, `c8.`, `c+16`); each dot lengthens by half
`r`	a rest (the same lengths)
`kN`	(noise track) a noise burst, N = 0-7: bit 2 white (else periodic), bits 1-0 the rate (0 high, 1 medium, 2 low, 3 follows voice 2); then a length

A voice may have only one note at a time (it is a chip with one square wave a voice); chords go in several voices.

4. The song format (SKM)

```
+0  'S' 'K'        the magic number
+2  version        1
+3  flags          bit 0: the song uses the noise voice
+4  loop target    a word: the offset to jump to at the end, or >FFFF for none
+6  length         a word: the file's length (mod 65536)
+8  commands ...
```

Command	Meaning
`>00`	the end: go to the loop target, or stop (and silence the chip)
`>01-3F`	hold the sound for that many frames (longer holds are chained)
`>40-7F`	write `command AND >3F` to the chip: the second byte of a tone's divider
`>80-FF`	write the byte to the chip: a latch, volume or noise byte

It is a register log, so the compiler does all the musical work and the player is 192 bytes of code; the price is that a song is bytes of chip traffic (a minute of a three-part tune with drums is a few kilobytes). A song may be up to 65,535 bytes; its pages must be consecutive, which the build does. The player reads at most 256 commands in a frame (a song with no hold would otherwise never return) and stops a song that does. To make songs another way, write the bytes yourself: the format is the whole interface.

5. Try it

[PC] the commands below. **[TI]** `examples/music` needs songs in SAMS pages, so it is a cartridge; on the TI use `programs/beep.a99` (workbench disk) for tones.

```
python3 tools/skmusic.py examples/music/songs/tune.mml --loop --wav /tmp/tune.wav --info     hear it on the PC first
python3 tools/skbuild.py examples/music                                                     build/music/music8.bin
python3 tests/test_kernel.py music                                                          the player, the compiler, the
model
```

`examples/music` plays two songs from a Text 80 screen: `1` the tune, `2` one A, `SPACE` pause or resume, `S` stop, `B` a beep with `SNDTONE`, `0` `SNDOFF`, `W` two seconds of work that never returns to the main loop and steps the music with `KIDLE`. Load `music8.bin` in js99er (or run it on the TI) and listen.

Decision: the default notation will be TXA's `.txm` (2 November 2026)

The owner has decided that TXA's `.txm` notation is the default for songs and sound effects. It is **not built yet**: the build needs TXA's notation specification and its compiler, which this tree does not have. Until it exists, songs are made with `tools/skmusic.py` as described above and the player's song format (bytes in consecutive pages) is unchanged. When `.txm` arrives it will compile to that same format (so existing songs and the player keep working), and `SFXPLAY` (effects that borrow voices from the song and give them back) will be added to the player.